

SC Assignment 2

Liam Erasmus

2026-03-03

Table of contents

Statistical Computing Assignment 2	3
1 Parallel Computing	4
1.1 Parallel Computing Prac	4
1.1.1 Question 1	4
1.1.2 Question 2	7
1.1.3 Question 3	8
1.1.4 Question 4	9
1.1.5 Question 5	9

Statistical Computing Assignment 2

Parallel Computing Practical

1 Parallel Computing

1.1 Parallel Computing Prac

1.1.1 Question 1

```
library(foreach)
library(kableExtra)
# Repeat 100 times: generate a sample from Exp(1) and compute mean and variance
one <- as.data.frame( foreach(i = 1:100, .combine = rbind) %do%{
  samp <- rexp(100, 1)
  c(
    x_bar <- mean(samp), # mean
    vari <- var(samp)) #variance
  #results = rbind(x_bar, vari)
})
)
kable(one,
  col.names = c("Sample Mean", "Sample Variance"))
```

	Sample Mean	Sample Variance
result.1	0.8714152	0.7521354
result.2	0.9007745	1.0861250
result.3	0.9817665	1.0788412
result.4	0.9776643	0.8558199
result.5	1.0552645	1.2736923
result.6	0.9730664	1.1128372
result.7	0.9704267	1.0404540
result.8	1.2070253	1.7680069
result.9	0.8793274	0.7692577
result.10	0.9360417	0.8624971
result.11	0.8894450	0.7290050
result.12	0.8995210	0.8730269
result.13	0.8639998	0.7475433

	Sample Mean	Sample Variance
result.14	0.9940936	0.8666937
result.15	1.0999457	1.2168721
result.16	0.9740340	0.7467526
result.17	0.8750013	0.5191186
result.18	1.0540959	1.0590907
result.19	0.8767371	0.9863702
result.20	1.0448048	1.1006080
result.21	1.0757477	0.8068678
result.22	0.9901474	0.7141006
result.23	0.9853428	1.0930254
result.24	0.9722165	0.9506244
result.25	1.1189061	0.8478567
result.26	0.9163315	0.5475347
result.27	0.9342699	0.7109015
result.28	1.0911831	1.0194530
result.29	1.0609187	1.0043657
result.30	0.8899904	1.3667040
result.31	0.8946633	0.7111101
result.32	0.9017306	0.6066180
result.33	0.9102943	0.8682052
result.34	1.0829446	0.8697344
result.35	0.8950641	0.7607361
result.36	1.0904344	1.0879141
result.37	0.9183281	0.7222073
result.38	1.0374381	1.2024204
result.39	1.0818753	1.1502532
result.40	1.1645319	1.5272780
result.41	0.8927617	0.8900670
result.42	1.0265550	0.9904864
result.43	0.9908834	0.7196794
result.44	0.9664763	1.0960861
result.45	1.0666273	1.2069539
result.46	1.0675310	0.9445255
result.47	0.9620989	1.6164029
result.48	1.0943466	0.9125692
result.49	0.8717107	0.6725500
result.50	1.1363052	1.6447675
result.51	0.9722173	0.7970708
result.52	1.1917640	1.7061165
result.53	1.0593833	1.1503293
result.54	1.0585623	1.5100407

	Sample Mean	Sample Variance
result.55	0.9287398	0.6331685
result.56	0.9646971	1.2290473
result.57	0.8917349	0.9074731
result.58	0.9305859	0.8153979
result.59	1.1316412	1.3349523
result.60	0.9732786	0.8517234
result.61	1.1696481	1.2302171
result.62	0.9527543	0.5911878
result.63	0.8758731	0.6947028
result.64	1.0392572	0.8424844
result.65	1.0516762	1.0625601
result.66	1.0366960	1.2888887
result.67	1.0243688	1.0441956
result.68	0.9006187	0.9540624
result.69	0.8136196	0.5568299
result.70	0.8872586	0.5757203
result.71	1.0840466	1.5920362
result.72	0.9804020	1.2323798
result.73	1.1254670	0.9511706
result.74	0.9582305	0.9274391
result.75	1.1745789	1.2862018
result.76	0.9763330	0.8766532
result.77	1.0289188	1.4741816
result.78	0.9568249	0.9814883
result.79	0.8608787	0.7297224
result.80	0.8671017	0.6054284
result.81	1.0249658	1.1636016
result.82	1.0005185	1.0714810
result.83	1.1161094	1.2306006
result.84	0.9151877	0.9184268
result.85	0.9163327	0.8692391
result.86	0.8846226	0.9022258
result.87	0.8905603	0.8754619
result.88	1.2075876	1.1923110
result.89	0.8950397	0.8440588
result.90	0.9986290	0.9986786
result.91	1.0604142	0.8275718
result.92	0.8207853	0.4660372
result.93	0.9595061	1.1097085
result.94	1.1271182	1.6487563
result.95	0.7483724	0.3584741

	Sample Mean	Sample Variance
result.96	1.0051138	0.7646706
result.97	1.3114925	1.5022510
result.98	1.0175044	1.0782060
result.99	1.2414616	1.4007870
result.100	0.7861668	0.6237684

1.1.2 Question 2

```
library(doParallel)
```

Loading required package: iterators

Loading required package: parallel

```
library(MASS)

x <- galaxies
n <- length(x)
B <- 200000

# Serial Bootstrap
t_serial <- system.time({
  med_serial <- replicate(B, median(sample(x, n, replace = TRUE)))
})["elapsed"]

t_serial
```

```
elapsed
2.517
```

```
# One Bootstrap per task (in parallel)

cl <- makeCluster(detectCores() - 1) # do not want my computer to crash
registerDoParallel(cl)

t_par_single <- system.time({
  med_par_single <- foreach(i = 1:B, .combine = c) %dopar% {
```

```

    median(sample(x, n, replace = TRUE))
  }
})["elapsed"]

stopCluster(cl)
t_par_single #MUCH SLOWER

```

```

elapsed
14.27

```

```

# 1000 Bootstraps per task
chunk_size <- 1000
nchunks <- 200

cl <- makeCluster(detectCores() - 1)
registerDoParallel(cl)

t_par_chunk <- system.time({
  med_par_chunk <- foreach(k = 1:nchunks, .combine = c) %dopar% {
    m <- if (k < nchunks) chunk_size else (B - (nchunks - 1) * chunk_size)
    replicate(m, median(sample(x, n, replace = TRUE)))
  }
})["elapsed"]

stopCluster(cl)

t_par_chunk # MUUUUCH FASTER

```

```

elapsed
0.486

```

1.1.3 Question 3

```

n <- 50 # sample size
mu_true <- 1 # true mean of Exp(1)
B <- 2000 # number of bootstrap samples
M <- 10000 # number of Monte Carlo simulations

cl3 <- makeCluster(detectCores() - 1) # again, do not crash

```

```

registerDoParallel(cl) # register cluster for parallel foreach

coverage_vec <- foreach(m = 1:M, .combine = c) %dopar% {

  x <- rexp(n, rate = 1)

  boot_means <- replicate(B, mean(sample(x, n, replace = TRUE)))

  ci <- quantile(boot_means, probs = c(0.025, 0.975), names = FALSE) # percentile bootstrap CI

  (ci[1] <= mu_true) && (mu_true <= ci[2]) # check whether the true mean lies inside the CI
}

stopCluster(cl3)

mean(coverage_vec) # estimated coverage probability

```

```
[1] 0.9244
```

1.1.4 Question 4

```

library(iterators)

set.seed(1234)

foreach(x = irnorm(3, count = 5), .combine = c) %do% {
  max(x) # max of each vector
}

```

```
[1] 1.0844412 0.5060559 -0.5466319 -0.4771927 0.9594941
```

1.1.5 Question 5

```

library(parallel)

R <- 50000
set.seed(1234)

```

```

# normal way
# Generate 3 samples of size 5 from N(0,1),
# take the maximum of each sample
t_rep <- system.time(replicate
                      (R, replicate
                       (3, max(rnorm(5))))))["elapsed"]

# with for each
# get the maximum of 3 samples
set.seed(1234)
t_for <- system.time(replicate(R,
  foreach(x = irnorm(3, count = 5), .combine = c) %do% max(x)
))["elapsed"]

# with parLapply
# distribute the 3 computations across cores
cl <- makeCluster(detectCores() - 1)
clusterSetRNGStream(cl, 1234)
t_par <- system.time(replicate(R,
  unlist(parLapply(cl, 1:3, function(i) max(rnorm(5))))
))["elapsed"]
stopCluster(cl)

c(replicate = t_rep, foreach = t_for, parLapply = t_par) # answers

```

```

replicate.elapsed    foreach.elapsed parLapply.elapsed
               0.390                24.159                15.274

```